

## Punto 4.11 - Diagnóstico con logs, métricas y herramientas

### Práctica

**Ejercicio:** Configurar el diagnóstico en el proyecto AceriaData. Añadir métodos al repositorio que midan el tiempo de ejecución de las consultas, cuenten el número de consultas y detecten las consultas lentas. Configurar un observador de diagnóstico. Analizar el plan de ejecución en SQL Server.

**Contexto del proyecto:** En el punto 4.10 se estudió la paginación eficiente, incluyendo Skip/Take y keyset pagination. En este punto se profundiza en el diagnóstico, que permite identificar problemas de rendimiento en EF Core. Esta técnica se usará en el punto 4.12 para las estrategias de optimización.

### Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

### Paso 2: Crear el observador de diagnóstico

Crear el archivo src/AceriaData.Infrastructure/Diagnostics/EfCoreDiagnosticObserver.cs:

```
using System.Diagnostics;

namespace AceriaData.Infrastructure.Diagnostics;

public class EfCoreDiagnosticObserver : IObservable<DiagnosticListener>
{
    private readonly Action<string> _log;

    public EfCoreDiagnosticObserver(Action<string> log)
    {
        _log = log;
    }

    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
        {
            listener.Subscribe(new EfCoreCommandObserver(_log));
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}

public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>
{
    private readonly Action<string> _log;

    public EfCoreCommandObserver(Action<string> log)
    {
        _log = log;
    }

    public void OnNext(KeyValuePair<string, object> value)
    {
        if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")
        {
            _log($"Comando ejecutado: {value.Value}");
        }
    }
}
```

```

    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}

```

Línea 1: using System.Diagnostics; → importa el espacio de nombres de diagnóstico.

Línea 3: namespace AceriaData.Infrastructure.Diagnostics; → declara el espacio de nombres.

Línea 5: public class EfCoreDiagnosticObserver : IObservable → declara el observador de listeners.

Línea 7: private readonly Action \_log; → campo para el delegado de logging.

Línea 9: public EfCoreDiagnosticObserver(Action log) → constructor.

Línea 11: \_log = log; → asigna el delegado al campo.

Línea 14: public void OnNext(DiagnosticListener listener) → implementa OnNext.

Línea 16: if (listener.Name == "Microsoft.EntityFrameworkCore") → comprueba el nombre del listener.

Línea 18: listener.Subscribe(new EfCoreCommandObserver(\_log)); → suscribe el observador interno.

Línea 22: public void OnError(Exception error) { } → implementa OnError.

Línea 23: public void OnCompleted() { } → implementa OnCompleted.

Línea 26: public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>> → declara el observador de eventos.

Línea 28: private readonly Action \_log; → campo para el delegado de logging.

Línea 30: public EfCoreCommandObserver(Action log) → constructor.

Línea 32: \_log = log; → asigna el delegado al campo.

Línea 35: public void OnNext(KeyValuePair<string, object> value) → implementa OnNext.

Línea 37: if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted") → comprueba el nombre del evento.

Línea 39: \_log(\$"Comando ejecutado: {value.Value}"); → registra el comando ejecutado.

Línea 43: public void OnError(Exception error) { } → implementa OnError.

Línea 44: public void OnCompleted() { } → implementa OnCompleted.

**Error común:** si el observador no comprueba el nombre del listener, se suscribe a todos los listeners del sistema, lo que puede degradar el rendimiento. Se debe comprobar el nombre.

### | Paso 3: Añadir los métodos de diagnóstico a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerTodasConDiagnostico();
    List<OrdenFabricacion> ObtenerPendientesConDiagnostico();
    int ContarOrdenesConDiagnostico();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerTodasConDiagnostico(); → método que carga todas las órdenes.

Línea 11: List ObtenerPendientesConDiagnostico(); → método que carga las órdenes pendientes.

Línea 12: int ContarOrdenesConDiagnostico(); → método que cuenta las órdenes.

Línea 14: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 15: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

**Error común:** si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

## | Paso 4: Implementar los métodos de diagnóstico en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```

public List<OrdenFabricacion> ObtenerTodasConDiagnostico()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPendientesConDiagnostico()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public int ContarOrdenesConDiagnostico()
{
    return _context.OrdenesFabricacion.Count();
}

```

Línea 1: public List ObtenerTodasConDiagnostico() → declara el método.

Línea 3: return \_context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta.

Línea 9: public List ObtenerPendientesConDiagnostico() → declara el método.

Línea 11: return \_context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .ToList(); → materializa la consulta.

Línea 18: public int ContarOrdenesConDiagnostico() → declara el método.

Línea 20: return \_context.OrdenesFabricacion.Count(); → cuenta las órdenes.

**Error común:** si no se aplica AsNoTracking, las entidades se registran en el Change Tracker y consumen memoria. Se debe aplicar en consultas de solo lectura.

## | Paso 5: Crear el caso de uso de diagnóstico

Crear el archivo src/AceriaData.Application/UseCases/DiagnosticoUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class DiagnosticoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public DiagnosticoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== DIAGNÓSTICO ===");

        DemostrarMedicionDeTiempo();
        DemostrarConteoDeConsultas();
        DemostrarDeteccionDeConsultasLentas();
    }

    private void DemostrarMedicionDeTiempo()
    {
        Console.WriteLine("\n--- Medición de tiempo ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarConteoDeConsultas()
    {
        Console.WriteLine("\n--- Conteo de consultas ---");

        var contador = 0;
        var cronometro = Stopwatch.StartNew();

        contador++;
    }
}
```

```

        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();

        contador++;
        var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();

        contador++;
        var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();

        cronometro.Stop();

        Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds}
ms");
        Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total:
{total}");
    }

    private void DemostrarDeteccionDeConsultasLentas()
    {
        Console.WriteLine("\n--- Detección de consultas lentas ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
        cronometro.Stop();

        var tiempoMs = cronometro.ElapsedMilliseconds;
        var umbralMs = 50;

        if (tiempoMs > umbralMs)
        {
            Console.WriteLine($"¡ALERTA! La consulta tardó {tiempoMs} ms, superando el umbral de
{umbralMs} ms.");
        }
        else
        {
            Console.WriteLine($"Consulta dentro del umbral: {tiempoMs} ms (umbral: {umbralMs} ms).");
        }
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class DiagnosticoUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo \_unidad; → campo de la unidad de trabajo.

Línea 10: public DiagnosticoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: \_unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== DIAGNÓSTICO ==="); → muestra la cabecera.

Línea 19: DemostrarMedicionDeTiempo(); → llama al método de medición de tiempo.

Línea 20: DemostrarConteoDeConsultas(); → llama al método de conteo de consultas.

Línea 21: DemostrarDeteccionDeConsultasLentas(); → llama al método de detección de consultas lentas.

Línea 24: private void DemostrarMedicionDeTiempo() → declara el método.

Línea 26: Console.WriteLine("\n--- Medición de tiempo ---"); → muestra la cabecera.

Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: var ordenes = \_unidad.Ordenes.ObtenerTodasConDiagnostico(); → carga las órdenes.

Línea 30: cronometro.Stop(); → detiene el cronómetro.

Línea 32: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 35: `private void DemostrarConteoDeConsultas()` → declara el método.

Línea 37: `Console.WriteLine("\n--- Conteo de consultas ---");` → muestra la cabecera.

Línea 39: `var contador = 0;` → inicializa el contador.

Línea 40: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 42: `contador++;` → incrementa el contador.

Línea 43: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → carga las órdenes.

Línea 45: `contador++;` → incrementa el contador.

Línea 46: `var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();` → carga las pendientes.

Línea 48: `contador++;` → incrementa el contador.

Línea 49: `var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();` → cuenta las órdenes.

Línea 51: `cronometro.Stop();` → detiene el cronómetro.

Línea 53: `Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 54: `Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total: {total}");` → muestra los datos.

Línea 57: `private void DemostrarDeteccionDeConsultasLentas()` → declara el método.

Línea 59: `Console.WriteLine("\n--- Detección de consultas lentas ---");` → muestra la cabecera.

Línea 61: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 62: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → carga las órdenes.

Línea 63: `cronometro.Stop();` → detiene el cronómetro.

Línea 65: `var tiempoMs = cronometro.ElapsedMilliseconds;` → obtiene el tiempo.

Línea 66: `var umbralMs = 50;` → declara el umbral.

Línea 68: `if (tiempoMs > umbralMs)` → comprueba si supera el umbral.

Línea 70: `Console.WriteLine($"¡ALERTA! La consulta tardó {tiempoMs} ms, superando el umbral de {umbralMs} ms.");` → muestra la alerta.

Línea 72: `else` → caso contrario.

Línea 74: `Console.WriteLine($"Consulta dentro del umbral: {tiempoMs} ms (umbral: {umbralMs} ms).");` → muestra el mensaje.

**Error común:** si el contador se incrementa manualmente, se puede olvidar incrementarlo en alguna consulta. Se debe usar un observador de diagnóstico para contar las consultas automáticamente.

## | Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<DiagnosticoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

**Error común:** si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

## | Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<DiagnosticoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = \_provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

**Error común:** si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

## | Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = \_provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List(); → crea la lista de órdenes.

Línea 8: for (int i = 1; i <= 20; i++) → repite veinte veces.

Línea 10: ordenes.Add(new OrdenFabricacion → crea una orden.

Línea 12: NumeroOrden = \$"OF-2024-{i:D4}", → asigna el número.

Línea 13: Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur", → asigna el cliente.

Línea 14: Estado = i % 3 == 0 ? "EnProceso" : "Pendiente", → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: `context.SaveChanges();` → inserta las órdenes.

**Error común:** si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

## Paso 9: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada consulta, el número de consultas ejecutadas y la detección de consultas lentas.

## Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== DIAGNÓSTICO ===

--- Medición de tiempo ---
Órdenes: 20 | Tiempo: 42 ms

--- Conteo de consultas ---
Consultas ejecutadas: 3 | Tiempo: 85 ms
Órdenes: 20 | Pendientes: 13 | Total: 20

--- Detección de consultas lentas ---
Consulta dentro del umbral: 42 ms (umbral: 50 ms).
```

La primera sección muestra el tiempo de la consulta. La segunda sección muestra el número de consultas ejecutadas y el tiempo total. La tercera sección muestra la detección de consultas lentas.

**Observaciones:** el tiempo de las consultas es bajo porque la tabla tiene pocas filas. El número de consultas es tres: una para cargar las órdenes, otra para cargar las pendientes y otra para contar. La detección de consultas lentas compara el tiempo con el umbral.

## Paso 11: Diagnosticar un error común

Modificar el método `DemostrarConteoDeConsultas` para no incrementar el contador manualmente:

```
private void DemostrarConteoDeConsultas()
{
    Console.WriteLine("\n--- Conteo de consultas ---");

    var contador = 0;
    var cronometro = Stopwatch.StartNew();

    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
    var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();

    cronometro.Stop();

    Console.WriteLine($"Consultas ejecutadas: {contador} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    Console.WriteLine($"Órdenes: {ordenes.Count} | Pendientes: {pendientes.Count} | Total: {total}");
}
```

Resultado esperado: el contador muestra cero porque no se ha incrementado. El número de consultas no es correcto.

Solución: usar un observador de diagnóstico para contar las consultas automáticamente.

```
private int _contadorConsultas;

private void DemostrarConteoDeConsultasConObservador()
{
    Console.WriteLine("\n--- Conteo de consultas con observador ---");

    _contadorConsultas = 0;
    var observer = new EfCoreDiagnosticObserver(message =>
    {
        if (message.StartsWith("Comando ejecutado")) _contadorConsultas++;
    });
    DiagnosticListener.AllListeners.Subscribe(observer);

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
    var total = _unidad.Ordenes.ContarOrdenesConDiagnostico();
    cronometro.Stop();

    Console.WriteLine($"Consultas ejecutadas: {_contadorConsultas} | Tiempo:
{cronometro.ElapsedMilliseconds} ms");
}
Resultado esperado con la solución: el contador muestra el número real de consultas ejecutadas.
```

## Errores comunes del ejercicio

| Error                            | Causa                                    | Solución                            |
|----------------------------------|------------------------------------------|-------------------------------------|
| Contador incorrecto              | Se incrementa manualmente                | Usar un observador de diagnóstico   |
| Logging sin filtrar              | Se registran todos los mensajes          | Filtrar por categoría y nivel       |
| Datos sensibles expuestos        | EnableSensitiveDataLogging en producción | Deshabilitar en producción          |
| Errores detallados en producción | EnableDetailedErrors en producción       | Deshabilitar en producción          |
| Observador sin filtrar           | Se suscribe a todos los listeners        | Comprobar el nombre del listener    |
| Medición imprecisa               | Se mide el tiempo de materialización     | Medir solo el tiempo de la consulta |

## Reto resuelto: Detectar consultas lentas con un observador de diagnóstico

**Reto:** Crear un observador de diagnóstico que detecte las consultas que tardan más de un umbral y las registre. Aplicarlo a las consultas del repositorio.

**Solución paso a paso:**

## Paso 1: Modificar el observador para detectar consultas lentas:

```
public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>
{
    private readonly Action<string> _log;
    private readonly long _umbralMs;

    public EfCoreCommandObserver(Action<string> log, long umbralMs)
    {
        _log = log;
        _umbralMs = umbralMs;
    }

    public void OnNext(KeyValuePair<string, object> value)
    {
        if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")
        {
            var tipo = value.Value.GetType();
            var propiedadDuracion = tipo.GetProperty("Duration");
            if (propiedadDuracion != null)
            {
                var duracion = (TimeSpan)propiedadDuracion.GetValue(value.Value!);
                if (duracion.TotalMilliseconds > _umbralMs)
                {
                    _log($"Consulta lenta detectada: {duracion.TotalMilliseconds} ms");
                }
            }
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}
```

Línea 1: `public class EfCoreCommandObserver : IObservable<KeyValuePair<string, object>>` → declara el observador.

Línea 3: `private readonly Action _log;` → campo para el delegado de logging.

Línea 4: `private readonly long _umbralMs;` → campo para el umbral.

Línea 6: `public EfCoreCommandObserver(Action log, long umbralMs)` → constructor.

Línea 8: `_log = log;` → asigna el delegado.

Línea 9: `_umbralMs = umbralMs;` → asigna el umbral.

Línea 12: `public void OnNext(KeyValuePair<string, object> value)` → implementa `OnNext`.

Línea 14: `if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")` → comprueba el evento.

Línea 16: `var tipo = value.Value.GetType();` → obtiene el tipo del evento.

Línea 17: `var propiedadDuracion = tipo.GetProperty("Duration");` → obtiene la propiedad `Duration`.

Línea 18: `if (propiedadDuracion != null)` → comprueba si existe.

Línea 20: `var duracion = (TimeSpan)propiedadDuracion.GetValue(value.Value!);` → obtiene la duración.

Línea 21: `if (duracion.TotalMilliseconds > _umbralMs)` → comprueba si supera el umbral.

Línea 23: `_log($"Consulta lenta detectada: {duracion.TotalMilliseconds} ms");` → registra la consulta lenta.

## Paso 2: Añadir la demostración en el caso de uso:

```
private void DemostrarDeteccionConObservador()
{
    Console.WriteLine("\n--- Detección de consultas lentas con observador ---");

    var observer = new EfCoreDiagnosticObserver(message => Console.WriteLine(message));
    DiagnosticListener.AllListeners.Subscribe(observer);

    var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();
    var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();
}
```

Línea 1: `private void DemostrarDeteccionConObservador()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Detección de consultas lentas con observador ---");` → muestra la cabecera.

Línea 5: `var observer = new EfCoreDiagnosticObserver(message => Console.WriteLine(message));` → crea el observador.

Línea 6: `DiagnosticListener.AllListeners.Subscribe(observer);` → suscribe el observador.

Línea 8: `var ordenes = _unidad.Ordenes.ObtenerTodasConDiagnostico();` → ejecuta la consulta.

Línea 9: `var pendientes = _unidad.Ordenes.ObtenerPendientesConDiagnostico();` → ejecuta la consulta.

## Paso 3: Llamar al método desde Ejecutar:

```
DemostrarDeteccionConObservador();
```

## Paso 4: Ejecutar dotnet run y verificar que las consultas lentas se detectan.

Resultado esperado: las consultas que superan el umbral se registran con el mensaje Consulta lenta detectada.

## Analogía final

El diagnóstico en una acería es como el sistema de control de calidad de la planta. Los logs son como el registro de cada operación: qué se hizo, cuándo y con qué parámetros. Las métricas son como los indicadores del panel de control: cuántas operaciones se han hecho, cuánto han tardado, cuántas han fallado. El plan de ejecución es como el plano que muestra cómo se ha ejecutado cada operación: si se ha usado el horno correcto, si se ha tardado más de lo previsto. Las herramientas de diagnóstico son como los instrumentos de medición: termómetros, manómetros, cronómetros. Sin diagnóstico, los problemas de rendimiento son invisibles. Con diagnóstico, se detectan antes de que afecten a la producción. Así funciona el diagnóstico en EF Core: se observa, se mide y se analiza para identificar problemas y aplicar soluciones.

## Resultado esperado

Al final del ejercicio, deberías haber:

Creado el observador de diagnóstico `EfCoreDiagnosticObserver`.

Añadido los métodos de diagnóstico a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `DiagnosticoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes.

Ejecutado las demostraciones de medición de tiempo, conteo de consultas y detección de consultas lentas.

Analizado la salida y comprobado el tiempo y el número de consultas.

Diagnosticado el error de incrementar el contador manualmente.

Creado el observador de consultas lentas.

---